软件工程专业 MySQL 实验指导书

适用专业：本科软件工程专业

实验目标：掌握 MySQL 数据库及官方工具 MySQL Workbench 的安装配置；理解数据库 ER 图建模原理及注意事项；熟练掌握数据表增删改查及跨表查询操作。

实验环境：Windows 10/11 操作系统、MySQL 8.0+、MySQL Workbench 8.0+

Navicat 参考视频：
https://www.bilibili.com/video/BV1Ni4y1c7sG?vd_source=ad1e914cfdd815e5a700bd10808fa5cb

mysql 参考视频：
https://www.bilibili.com/video/BV1dbaMz7ECB?vd_source=ad1e914cfdd815e5a700bd10808fa5cb

第一章 MySQL 与 Navicat 安装配置指导

1.1 MySQL 安装（以 Windows 10 为例）

1.1.1 安装前准备

1. 访问 MySQL 官方下载地址（https://dev.mysql.com/downloads/mysql/），选择“MySQL Installer for Windows”，下载对应版本（推荐 8.0.44）。

2. 确认操作系统位数（32 位/64 位），选择匹配的安装包；若未安装 Microsoft Visual C++ Redistributable，需提前下载安装。

1.1.2 详细安装步骤

1. 双击下载的安装包，启动安装程序，选择“Custom”自定义安装（推荐，可自主选择组件）。

2. 在组件选择界面，展开“MySQL Server 8.0.x”勾选核心组件，点击“Next”。

3. 点击“Execute”执行组件下载与安装，等待完成后点击“Next”。

4. 进入配置界面，选择“Standalone MySQL Server/Classic MySQL Replication”，点击“Next”。

5. 配置端口号（默认 3306，若被占用可修改，需记录端口号），勾选“Open Firewall

port for network access”，点击“Next”。

6. 选择身份验证方式，推荐“Use Legacy Authentication Method (Retain MySQL 5.x Compatibility)”（兼容旧版本，降低后续连接问题），点击“Next”。

7. 设置 root 用户密码（至关重要，务必牢记，推荐复杂度高的密码），可点击“Add User”创建普通用户（可选，用于后续实验权限管理练习），点击“Next”。

8. 配置 MySQL 服务名称（默认 MySQL80），勾选“Start the MySQL Server at System Startup”（开机自启，方便使用），点击“Next”。

9. 点击“Execute”执行配置，全部完成后点击“Finish”，MySQL 安装结束。

1.1.3 安装验证

1. 按下 Win+R，输入“cmd”打开命令提示符。

2. 进入 MySQL 安装目录的 bin 文件夹（示例路径：cd C:\Program Files\MySQL\MySQL Server 8.0\bin）。

3. 输入命令：mysql -u root -p，按回车后输入安装时设置的 root 密码，若出现“mysql>”提示符，说明安装成功。

1.2 Navicat 下载与安装步骤（Windows 10/11）

1.2.1 安装前准备

1. 访问 Navicat 中文官网：https://www.navicat.com.cn/
2. 选择版本：
   - 免费版：Navicat Premium Lite（原社区版，支持 MySQL、MariaDB 等主流数据库，无试用期限制）
   - 付费版：Navicat Premium（全数据库支持，含 14 天试用期）
   - 专用版：Navicat for MySQL（仅管理 MySQL/MariaDB）
3. 确认系统位数：Windows 10/11 主流为 64 位，选择对应 64 位安装包
4. 关闭杀毒软件 / 防火墙（临时），避免拦截安装文件

1.2.2 详细下载步骤

1. 官网首页点击顶部【下载】→ 选择对应版本（如 Navicat Premium Lite）
2. 进入下载页，选择【Windows (64 bit)】→ 点击【直接下载】
3. 安装包保存至浏览器默认下载目录（如 “下载” 文件夹），文件名为

navicat17_premium_lite_cs_x64.exe

1.2.3 详细安装步骤

1. 找到下载的 .exe 安装包，右键 → 以管理员身份运行（关键，避免权限不足）
2. 弹出用户账户控制，点击【是】允许程序修改计算机
3. 选择安装语言：默认【简体中文】→ 点击【确定】
4. 欢迎界面：点击【下一步】
5. 许可协议：勾选【我接受许可协议中的条款】→ 点击【下一步】
6. 选择安装路径：
7. 默认：C:\Program Files\PremiumSoft\Navicat Premium Lite
8. 建议修改为非系统盘（如 D:\Navicat Premium Lite）→ 点击【浏览】自定义 → 【下一步】
9. 选择开始菜单文件夹：默认即可 → 点击【下一步】
10. 附加任务：勾选【创建桌面图标】（方便快速启动）→ 点击【下一步】
11. 准备安装：确认配置无误 → 点击【安装】，等待 2-3 分钟完成文件复制
12. 安装完成：勾选【运行 Navicat Premium Lite】→ 点击【完成】
13. 启动后弹出登录界面：
    - 免费版（Premium Lite）：点击【创建 Navicat ID】注册账号，登录后即可永久使用
    - 付费版：可选择【试用 14 天】，或输入激活码激活
14. 进入主界面，安装完成。

1.2.4 安装验证

1. 双击桌面 Navicat 图标，正常启动无报错
2. 点击【连接】→ 选择【MySQL】，输入本地 MySQL 信息（主机：localhost，端口：3306，用户名：root，密码：安装时设置的密码）
3. 点击【测试连接】，提示【连接成功】，说明安装与配置正常

第二章 数据库增删改查及跨表查询

数据库操作通常包括数据库的创建与删除；数据库表的创建与删除；以及数据表单的增删改查（CRUD）和多表的跨表查询。这些是软件开发中数据库操作的基础。

2.1 创建和删除数据库

2.1.1 新建数据库（CREATE DATABASE）

语法格式:

```
-- 方式一：基础创建（推荐，简洁明了）
CREATE DATABASE 数据库名;

-- 方式二：创建时指定字符集（避免中文乱码，生产环境推荐）
CREATE DATABASE 数据库名 CHARACTER SET 字符集 COLLATE 校对规则;
```

示例:

```
-- 基础创建：新建名为 library 的图书馆管理数据库
CREATE DATABASE library;

-- 指定字符集创建：新建名为 library 的数据库，指定 UTF8 字符集（兼容 MySQL 5.x）
CREATE DATABASE library CHARACTER SET utf8 COLLATE utf8_general_ci;
```

2.1.2 删除数据库（DROP DATABASE）

语法格式:

```
-- 方式一：基础删除（注意：删除不可逆，需谨慎）
DROP DATABASE 数据库名;

-- 方式二：安全删除（仅当数据库存在时删除，避免不存在时报错）
DROP DATABASE IF EXISTS 数据库名;
```

示例:

```
-- 基础删除：删除名为 library 的数据库（若不存在会报错）
DROP DATABASE library;

-- 安全删除：仅当 library 数据库存在时删除（推荐使用）
DROP DATABASE IF EXISTS library;
```

2.2 创建数据表

2.2.1 新建数据库表（CREATE TABLE）

语法格式：

```
-- 方式一：基础格式（指定列名、数据类型、约束）
CREATE TABLE 表名 (
    列名 1 数据类型 [约束条件],
    列名 2 数据类型 [约束条件],
    ...
    [表级约束，如主键、外键等] );

-- 方式二：安全创建（仅当表不存在时创建，避免报错）
CREATE TABLE IF NOT EXISTS 表名 (
    列名 1 数据类型 [约束条件],
    列名 2 数据类型 [约束条件],
    ... );
```

关键说明：

- 常用约束：`PRIMARY KEY`（主键，唯一标识行）、`NOT NULL`（非空）、`UNIQUE`（唯一）、`DEFAULT`（默认值）、`FOREIGN KEY`（外键，关联其他表）；
- 常用数据类型：`INT`（整数）、`VARCHAR(长度)`（字符串）、`DATE`（日期）、`DATETIME`（日期时间）等。

示例：新建 reader、book、borrow 三张表

```
-- 新建读者表（reader）
CREATE TABLE IF NOT EXISTS reader (
```

```
    reader_id INT PRIMARY KEY AUTO_INCREMENT, -- 读者 ID，主键自增
    reader_name VARCHAR(50) NOT NULL, -- 读者姓名，非空
    reader_gender VARCHAR(10), -- 读者性别
    reader_phone VARCHAR(20) UNIQUE, -- 读者手机号，唯一
    register_date DATE -- 注册日期 );

-- 新建图书表（book）
CREATE TABLE IF NOT EXISTS book (
    book_id INT PRIMARY KEY AUTO_INCREMENT, -- 图书 ID，主键自增
    book_name VARCHAR(100) NOT NULL, -- 图书名称，非空
    book_author VARCHAR(50), -- 作者
    publisher VARCHAR(50), -- 出版社
    stock_quantity INT DEFAULT 0, -- 库存数量，默认 0
    publish_date DATE -- 出版日期 );

-- 新建借阅表（borrow），关联 reader 和 book 表
CREATE TABLE IF NOT EXISTS borrow (
    borrow_id INT PRIMARY KEY AUTO_INCREMENT, -- 借阅 ID，主键自增
    reader_id INT NOT NULL, -- 关联的读者 ID
    book_id INT NOT NULL, -- 关联的图书 ID
    borrow_date DATETIME DEFAULT NOW(), -- 借阅时间，默认当前时间
    return_date DATETIME, -- 归还时间
    -- 外键约束：关联读者表
    FOREIGN KEY (reader_id) REFERENCES reader(reader_id),
    -- 外键约束：关联图书表
    FOREIGN KEY (book_id) REFERENCES book(book_id) );
```

2.2.2 删除数据库表（DROP TABLE）

语法格式：

```
-- 方式一：基础删除（注意：删除不可逆，表中数据会全部清空）
DROP TABLE 表名;
```

```
-- 方式二：安全删除（仅当表存在时删除，避免不存在时报错）
DROP TABLE IF EXISTS 表名;
-- 方式三：批量删除多个表
DROP TABLE IF EXISTS 表名1, 表名2, ...;
```

示例:

```
-- 单表删除：安全删除 borrow 表（先删关联表，避免外键约束报错）
DROP TABLE IF EXISTS borrow;
-- 批量删除：一次性删除三张表（需注意顺序，先删有外键的 borrow）
DROP TABLE IF EXISTS borrow, book, reader;
```

2.3 单表增删改查（CRUD）

2.3.1 新增数据（INSERT）

语法格式:

```
-- 方式一：指定列名插入（推荐，顺序可自定义）
INSERT INTO 表名 (列名1, 列名2, ...) VALUES (值1, 值2, ...);

-- 方式二：不指定列名（需按表中列的顺序插入所有值）
INSERT INTO 表名 VALUES (值1, 值2, ...);
```

示例:

```
-- 向 reader 表插入 2 条读者数据
INSERT INTO reader (reader_name, reader_gender, reader_phone, register_date)
VALUES ('张三', '男', '13800138001', '2023-01-15'),
       ('李四', '女', '13900139001', '2023-02-20');

-- 向 book 表插入 3 条图书数据
INSERT INTO book (book_name, book_author, publisher, publish_date, stock_quantity)
VALUES ('MySQL 必知必会', 'Ben Forta', '人民邮电出版社', '2020-05-01', 10),
       ('Java 编程思想', 'Bruce Eckel', '机械工业出版社', '2021-03-10',
```

```
8),
       ('Python 编程：从入门到实践', '埃里克·马瑟斯', '人民邮电出版社',
'2019-12-01', 15);

-- 向 borrow 表插入 2 条借阅数据（假设张三借了 MySQL 和 Java 的书，未归还）
INSERT INTO borrow (reader_id, book_id, borrow_date)
VALUES (1, 1, '2023-10-01'),
       (1, 2, '2023-10-01');
```

注意：1. 字符串类型的值需用单引号包裹；2. 日期类型的值格式需为'YYYY-MM-DD'；3. 非空列必须插入值；4. 自增主键无需插入值，数据库会自动生成。

2.3.2 查询数据（SELECT）

基础语法：

```
-- 查询指定列
SELECT 列名 1, 列名 2, ... FROM 表名;

-- 查询所有列（不推荐，效率低，仅测试用）
SELECT * FROM 表名;

-- 条件查询（WHERE）
SELECT 列名 FROM 表名 WHERE 条件;

-- 排序查询（ORDER BY，ASC 升序，DESC 降序，默认 ASC）
SELECT 列名 FROM 表名 ORDER BY 列名 排序方式;

-- 去重查询（DISTINCT）
SELECT DISTINCT 列名 FROM 表名;
```

示例：

```
-- 1. 查询所有读者的姓名和联系电话
SELECT reader_name, reader_phone FROM reader;

-- 2. 查询馆藏数量大于 10 的图书信息，按馆藏数量降序排序
SELECT * FROM book WHERE stock_quantity > 10 ORDER BY stock_quantity
DESC;

-- 3. 查询 2023 年 2 月及以后注册的读者
SELECT * FROM reader WHERE register_date >= '2023-02-01';

-- 4. 查询所有出版社（去重）
SELECT DISTINCT publisher FROM book;
```

2.3.3 修改数据（UPDATE）

语法格式：

```
UPDATE 表名 SET 列名1=值1, 列名2=值2, ... WHERE 条件;
```

警告：UPDATE 语句必须加 WHERE 条件，否则会修改表中所有记录！

示例：

```
-- 1. 将读者张三的联系电话改为13800138002
UPDATE reader SET reader_phone = '13800138002' WHERE reader_name = '张三';

-- 2. 将《MySQL 必知必会》的馆藏数量减少1（被借阅一本）
UPDATE book SET stock_quantity = stock_quantity - 1 WHERE book_name = 'MySQL 必知必会';

-- 3. 标记张三借阅的《MySQL 必知必会》已归还（更新归还日期）
UPDATE borrow SET return_date = '2023-10-15' WHERE reader_id = 1 AND book_id = 1;
```

2.3.4 删除数据（DELETE）

语法格式：

```
DELETE FROM 表名 WHERE 条件;
```

警告：1. DELETE 语句必须加 WHERE 条件，否则会删除表中所有记录；2. 若表中存在外键关联，需先删除关联表中的数据，再删除当前表数据（或开启级联删除）。

示例：

```
-- 1. 删除读者李四的记录（假设李四无借阅记录）
DELETE FROM reader WHERE reader_id = 2;

-- 2. 删除张三借阅《Java 编程思想》的记录（同时需将图书馆藏数量加1）
UPDATE book SET stock_quantity = stock_quantity + 1 WHERE book_id = 2;
DELETE FROM borrow WHERE reader_id = 1 AND book_id = 2;
```

2.4 跨表查询（多表查询）

软件开发中，数据往往分布在多个表中，需通过跨表查询获取完整数据。核心方法包括**内连接（INNER JOIN）**、**左连接（LEFT JOIN）**、**右连接（RIGHT JOIN）**，其中内连接和左连接最常用。

核心语法：

```
-- 内连接（只查询两个表中匹配的记录）
SELECT 列名 FROM 表1 INNER JOIN 表2 ON 表1.外键列 = 表2.主键列 WHERE 条件;

-- 左连接（查询表1中所有记录，以及表2中匹配的记录，表2无匹配则为NULL）
SELECT 列名 FROM 表1 LEFT JOIN 表2 ON 表1.外键列 = 表2.主键列 WHERE 条件;

-- 多表连接（3个及以上表，依次连接）
SELECT 列名 FROM 表1 JOIN 表2 ON 表1.外键1 = 表2.主键 JOIN 表3 ON 表2.外键2 = 表3.主键 WHERE 条件;
```

2.4.1 两表连接示例（reader 与 borrow）

需求 1：查询所有借阅记录，显示读者姓名、借阅日期、归还日期。

```
-- 内连接（只显示有借阅记录的读者）
SELECT r.reader_name, b.borrow_date, b.return_date
FROM reader r
INNER JOIN borrow b
ON r.reader_id = b.reader_id;

-- 左连接（显示所有读者，无借阅记录的读者归还日期为NULL）
SELECT r.reader_name, b.borrow_date, b.return_date
FROM reader r
LEFT JOIN borrow b
ON r.reader_id = b.reader_id;
```

说明：表名后加别名（如 reader r），可简化 SQL 语句。

2.4.2 三表连接示例（reader、borrow 与 book）

需求 2：查询所有已归还图书的记录，显示读者姓名、图书名称、借阅日期、归还日期，按借阅日期降序排序。

```
SELECT
  r.reader_name,
  bk.book_name,
  br.borrow_date,
  br.return_date
FROM reader r
INNER JOIN borrow br ON r.reader_id = br.reader_id
INNER JOIN book bk ON br.book_id = bk.book_id
```

```
WHERE br.return_date IS NOT NULL  -- 已归还
ORDER BY br.borrow_date DESC;
```

需求 3：查询“人民邮电出版社”出版的图书被哪些读者借阅过，显示读者姓名、图书名称、借阅日期。

```
   SELECT
    r.reader_name,
    bk.book_name,
    br.borrow_date
FROM book bk
INNER JOIN borrow br ON bk.book_id = br.book_id
INNER JOIN reader r ON br.reader_id = r.reader_id
WHERE bk.publisher = '人民邮电出版社';
```

第三章 ER 图到数据库表的转化

ER 图建模完成后，需将其转化为实际的数据库表结构，核心原则是：实体对应表，属性对应列，主键对应表的主键，关系通过外键实现。以下针对不同关系场景详细讲解转化方法。

3.1 基础转化规则

- 每个实体对应一个数据表，表名与实体名一致（推荐英文小写，如 reader、book）。
- 实体的每个属性对应表的一个列，列名与属性名一致（遵循命名规范），同时指定合适的数据类型（如 VARCHAR、INT、DATE 等）。
- 实体的主键对应表的主键（PRIMARY KEY），确保唯一性和非空性。
- 实体间的关系通过外键（FOREIGN KEY）实现，外键列引用关联表的主键列。

3.2 不同关系场景的转化方法

3.2.1 一对多关系（1:N）

场景说明：一个实体的一条记录可对应另一个实体的多条记录，反之则只能对应一条。如“班级”与“学生”（1 个班级有多个学生，1 个学生属于 1 个班级）。

转化方法：在“多”的一方的表中添加一个外键列，引用“一”的一方的表的主键。

示例：班级（class）与学生（student）

- 班级实体（class）：班级 ID（class_id，主键）、班级名称（class_name）、班主任（class_teacher）；
- 学生实体（student）：学生 ID（stu_id，主键）、姓名（stu_name）、性别

（stu_gender）、年龄（stu_age）、班级 ID（class_id，外键，引用 class 表的 class_id）。

建表 SQL：

```
    -- 班级表（一的一方）
CREATE TABLE class (
    class_id INT AUTO_INCREMENT PRIMARY KEY,
    class_name VARCHAR(50) NOT NULL,
    class_teacher VARCHAR(30) NOT NULL
);

-- 学生表（多的一方，添加外键）
CREATE TABLE student (
    stu_id INT AUTO_INCREMENT PRIMARY KEY,
    stu_name VARCHAR(30) NOT NULL,
    stu_gender CHAR(2) DEFAULT '男',
    stu_age INT,
    class_id INT,
    -- 外键约束
    FOREIGN KEY (class_id) REFERENCES class(class_id)
    -- 可选：级联操作（删除班级时删除对应学生，或设置为 NULL）
    ON DELETE CASCADE ON UPDATE CASCADE
);
```

说明：ON DELETE CASCADE 表示当删除 class 表中的记录时，自动删除 student 表中对应 class_id 的记录；ON UPDATE CASCADE 表示当 class 表的 class_id 更新时，student 表的 class_id 同步更新，可根据业务需求选择是否添加。

3.2.2 多对多关系（M:N）

场景说明：两个实体的一条记录均可对应对方的多条记录。如“学生”与“课程”（1 个学生可选多门课，1 门课有多个学生选）、“读者”与“图书”（1 个读者可借多本书，1 本书可被多个读者借）。

转化方法：无法直接通过外键实现，需创建一个“中间表”（也叫关联表），中间表中包含两个外键列，分别引用两个实体对应表的主键；中间表的主键可设为两个外键的组合主键，或单独设置自增主键。

示例：学生（student）与课程（course）

- 学生实体（student）：学生 ID（stu_id，主键）、姓名（stu_name）、专业（stu_major）；
- 课程实体（course）：课程 ID（course_id，主键）、课程名称（course_name）、

学分（course_credit）；

- 中间表（stu_course）：学生 ID（stu_id，外键，引用 student.stu_id）、课程 ID（course_id，外键，引用 course.course_id）、成绩（score，可选，属于关系属性）；组合主键（stu_id, course_id）。

建表 SQL：

```
    -- 学生表
CREATE TABLE student (
    stu_id INT AUTO_INCREMENT PRIMARY KEY,
    stu_name VARCHAR(30) NOT NULL,
    stu_major VARCHAR(50) NOT NULL
);

-- 课程表
CREATE TABLE course (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    course_credit INT NOT NULL
);

-- 中间表（学生-课程关联表）
CREATE TABLE stu_course (
    stu_id INT NOT NULL,
    course_id INT NOT NULL,
    score DECIMAL(5,2), -- 成绩，保留 2 位小数
    -- 组合主键
    PRIMARY KEY (stu_id, course_id),
    -- 外键约束
    FOREIGN KEY (stu_id) REFERENCES student(stu_id) ON DELETE
CASCADE,
    FOREIGN KEY (course_id) REFERENCES course(course_id) ON DELETE
CASCADE
);
```

3.2.3 一对一关系（1:1）

场景说明：两个实体的一条记录只能对应对方的一条记录。如“用户”与“用户详情”（1 个用户有 1 份详情，1 份详情属于 1 个用户）、“员工”与“工牌”（1 个员工有 1 个工牌，1 个工牌对应 1 个员工）。

转化方法：有两种常用方式，根据业务需求选择：

1. **方式一：共享主键**：让两个表的主键相同，即一个表的主键同时作为另一个表的主键和外键。适用于两个实体紧密关联、生命周期一致的场景（如用户注册时必创建用户详情）。

2. **方式二：外键唯一**：在其中一个表中添加外键列，引用另一个表的主键，并给外键列添加唯一约束（UNIQUE）。适用于两个实体可独立存在的场景（如员工入职后可

能暂未发放工牌)。

示例：用户（user）与用户详情（user_detail）

方式一：共享主键建表 SQL：

```
-- 用户表（主表）
CREATE TABLE user (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(30) NOT NULL UNIQUE,
    password VARCHAR(50) NOT NULL,
    create_time DATE NOT NULL
);

-- 用户详情表（从表，共享主键）
CREATE TABLE user_detail (
    user_id INT PRIMARY KEY,
    real_name VARCHAR(30),
    phone VARCHAR(11),
    address VARCHAR(200),
    -- 外键约束
    FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE
);
```

方式二：外键唯一建表 SQL：

```
-- 用户表（主表）
CREATE TABLE user (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(30) NOT NULL UNIQUE,
    password VARCHAR(50) NOT NULL,
    create_time DATE NOT NULL
);

-- 用户详情表（从表，外键唯一）
CREATE TABLE user_detail (
    detail_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE, -- 外键+唯一约束
    real_name VARCHAR(30),
    phone VARCHAR(11),
    address VARCHAR(200),
    -- 外键约束
    FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE
);
```

3.3 转化注意事项

- **数据类型匹配**：外键列的数据类型必须与引用的主键列完全一致（如主键为 INT，外键也必须为 INT；主键为 VARCHAR(20)，外键也必须为 VARCHAR(20)）。
- **外键约束谨慎使用**：外键约束能保证数据完整性，但会降低数据库操作效率（尤其是批量插入/删除数据时）；开发环境可开启，生产环境若需高性能，可通过业务代码保证数据完整性，关闭外键约束。
- **中间表设计**：多对多关系的中间表除了外键列，还可包含关系本身的属性（如学生选课的成绩、读者借阅的日期）。
- **表名与列名规范**：统一采用小写字母，多个单词用下划线分隔（蛇形命名法），避免使用关键字，确保可读性。
- **非空约束设置**：根据业务需求设置 NOT NULL 约束（如学生姓名、课程名称必须非空），避免出现无效数据。